

CLamb

Lexical Specifications

1. Token set / Specifications

Category	Class Part	Value Part	Expected Output Format
Data Types	data_type	int, float, char, bool, string, void, auto	<data_type, float> <data_type, int>
Keyword if	kw_if	if	<kw_if, if>
Keyword else	kw_else	else	<kw_else, else>
Keyword for	kw_for	for	<kw_for, for>
Keyword while	kw_while	while	<kw_while, while>
Keyword return	kw_return	return	<kw_return, return>
Keyword lamb	kw_lamb	lamb	<kw_lamb, lamb>
Keyword null	kw_null	null	<kw_null, null>
Keyword print	kw_print	print	<kw_print, print>
Keyword malloc	kw_malloc	malloc	<kw_malloc, malloc>
Keyword free	kw_free	free	<kw_free, free>
Identifiers	identifier	User-defined string	<identifier, total> <identifier, 2sum>
Integer Literals	integer_literal	Numeric String	<integer_literal, 7>
Float Literals	float_literal	Decimal String	<float_literal, 3.14>
Char Literals	char_literal	Single Character	<char_literal, a>
String Literals	string_literal	Text enclosed in “	<string_literal, “hi”>
Boolean Literals	boolean_literal	true, false	<boolean_literal, “true”>

Additive Operators	add_op	+, -	<add_op, +>
Multiplicative Operators	mul_op	/, %	<mul_op, %>
Asterisk	asterisk	*	<asterisk, *>
Logical And	logical_and	AND	<logical_and, AND>
Logical Or	logical_or	OR	<logical_or, OR>
Logical Not	logical_not	NOT	<logical_not, NOT>
Relational Lesser Than	relational_less	<	<relational_less, <>
Relational Greater Than	relational_greater	>	<relational_greater, >>
Relational Less Than or Equal To	relational_lesseq	<=	<relational_lesseq, <=>
Relational Greater Than or Equal To	relational_greatereq	>=	<relational_greatereq, >=>
Relational Equal To	relational_eq	==	<relational_eq, ==>
Relational Not Equal To	relational_noteq	!=	<relational_noteq, !=>
Assignment	assignment_op	=	<assignment_op, =>
Address Operator	address_op	&	<address_op, &>
Left Parenthesis	l_paren	(	<l_paren, (>
Right Parenthesis	r_paren	)	<r_paren,)>
Left Brace	l_brace	{	<l_brace, {>
Right Brace	r_brace	}	<r_brace, }>
Left Bracket	l_bracket	[	<l_bracket, [>
Right Bracket	r_bracket	]	<r_bracket,]>
Semicolon	semicolon	;	<semicolon, ;>

Comma	comma	,	<comma, ,>
Comment	comment	//	<comment, //>

2. DFA Transition Table

Character Classes

- Letters (l) = a-z, A-Z
- Underscore = _
- Digits (d) = 0-9
- Whitespace (WS) = (space, tabs, newline)
- Other = Any character not explicitly matched in the current state.

Reserved Keywords Table:

Lexeme Set	Emitted Token
if	<kw_if, if>
else	<kw_else, else>
for	<kw_for, for>
while	<kw_while, while>
return	<kw_return, return>
lamb	<kw_lamb, lamb>
null	<kw_null, null>
print	<kw_print, print>
malloc	<kw_malloc, malloc>
free	<kw_free, free>
int, float, char, bool, string, void, auto	<data_type, value>
true, false	<boolean_literal, value>
AND	<logical_and, AND>
OR	<logical_or, OR>
NOT	<logical_not, NOT>

anything else	<identifier, value>
---------------	---------------------

Current State	Input Character Class	Next State	Accepting State?
S0(Start)	Letter	S1	No
S0	Digit	S2	No
S0	_	S1	No
S0	"	S5	No
S0	'	S6	No
S0	=	S8	No
S0	<	S9	No
S0	>	S10	No
S0	!	S11	No
S0	/	S12	No
S0	+, -	S_AddOp	Yes (<add_op, value>)
S0	*	S_Asterisk	Yes (<asterisk, *>)
S0	%	S_MulOp	Yes (<mul_op, %>)
S0	&	S_Ampersand	Yes (<address_op, &>)
S0	(	S_LParen	Yes (<l_paren, (>)
S0	)	S_RParen	Yes (<r_paren,)>)
S0	{	S_LBrace	Yes (<l_brace, {>)
S0	}	S_RBrace	Yes (<r_brace, }>)
S0	[	S_LBracket	Yes (<l_bracket, [>)
S0	]	S_RBracket	Yes (<r_bracket,]>)
S0	;	S_Semicolon	Yes (<semicolon, ;>)

S0	,	S_Comma	Yes (<comma, >)
S0	WS	S0	No
S0	Other	S_Error	Error State
S1(Identifier)	Letter, Digit, _	S1	No
S1	Other	S1(Accept)	Yes. reserved-word lookup: Maps to specific <kw_*, value>, <boolean_literal, value>, <logical_and, AND>, <logical_or, OR>, <logical_not, NOT>, <data_type, value>, or falls back to <identifier, value>. Token not consumed, pushback
S2(Digit-Initial)	Digit	S2	No
S2	_	S1	No
S2	. (fullstop)	S3	No
S2	Letter	S1	No
S2	Other	S2(Accept)	Yes (<integer_literal, value>), not consumed, pushback
S3 (Float dot)	Digit	S4	No
S3	Other	S_Error	Error State
S4(Float)	Digit	S4	No
S4	Letter	S_Error	Error State
S4	Other	S4(Accept)	Yes (<float_literal, value>), not consumed, pushback
S5(String)	"	S5(Accept)	Yes (<string_literal, value>)
S5	Other (including WS)	S5	No
S6(Char Start)	Any char (except ')	S7	No

S6	' (Immediately)	S_Error	Error State
S7(Char End)	'	S7(Accept)	Yes (<char_literal, value>)
S7	Other	S_Error	Error State
S8(=)	=	S8_Eq(Accept)	Yes (<relational_eq, == >), token consumed
S8	Other	S8_Assign(Accept)	Yes (<assignment_op, = >), not consumed, pushback
S9(<)	=	S9_Eq(Accept)	Yes (<relational_lesseq, <= >), token consumed
S9	Other	S9_Lt(Accept)	Yes (<relational_less, < >) not consumed, pushback
S10(>)	=	S10_Eq(Accept)	Yes (<relational_greatereq, >= >), token consumed
S10	Other	S10_Gt(Accept)	Yes (<relational_greater, > >) not consumed, pushback
S11(!)	=	S11_Eq(Accept)	Yes (<relational_noteq, !=>) token consumed
S11	Other	S_Error	Error State, token not consumed, pushback
S12(/)	/	S13	No (Comment Starts)
S12	Other	S12(Accept)	Yes (<mul_op, />), not consumed, pushback
S13(Comment)	\n (Newline)	S0	No
S13	Other	S13	No

S_Error	Any	S_Error	Consume and discard character; emit error diagnostic once on entry
S_Error	WS	S0	Resume normal tokenizing
S_Error	Recognizable start char (Letter, Digit, " , ' , operator chars)	S0	Resume normal tokenizing without consuming this character (re-lex it as a fresh token)

3. The Source Code & Test Inputs

The Source code and test inputs for the Lexical Analyzer are attached in the LMS submission. I have also attached the link to my GitHub repository for you to check the files and the outputs produced.

Test Scripts:

4_test_input.clamb
behaviour)

(verifies a valid script and tokenization

Error_test_input.clamb

(tests how the analyzer handles invalid statements)

run_tests.py

verify_table.py

[AmanTheDrummer/CLamb: A custom language built on the basics of C to understand how compilers work. This language introduces the concept of lambda functions to C.](#)